Laboratorio de Orquestación de Kubernetes en AWS con Amazon EKS

Yennyfer Andrea Gómez Hoyos

Docente: Ing. Mario German Castillo R.

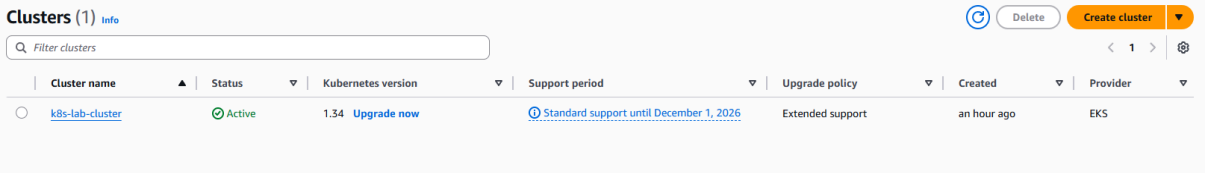
Cloud 2 Betek

Orquestación de Kubernetes en AWS con Amazon EKS

1. Creación del Clúster

```
2026-04-10 21:41:22 [ℹ]  successfully created addon: kube-proxy
2026-04-10 21:41:22 [ℹ]  creating addon: coredns
2026-04-10 21:41:23 [ℹ]  successfully created addon: coredns
2026-04-10 21:43:25 [ℹ]  building managed nodegroup stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:43:26 [ℹ]  deploying stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:43:26 [ℹ]  waiting for CloudFormation stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:43:57 [ℹ]  waiting for CloudFormation stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:44:44 [ℹ]  waiting for CloudFormation stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:45:52 [ℹ]  waiting for CloudFormation stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:46:26 [ℹ]  waiting for CloudFormation stack "eksctl-k8s-lab-cluster-nodegroup-standard-workers"
2026-04-10 21:46:26 [ℹ]  waiting for the control plane to become ready
2026-04-10 21:46:27 [✓]  saved kubeconfig as "/home/andrea/.kube/config"
2026-04-10 21:46:27 [ℹ]  no tasks
2026-04-10 21:46:27 [✓]  all EKS cluster resources for "k8s-lab-cluster" have been created
2026-04-10 21:46:28 [ℹ]  nodegroup "standard-workers" has 2 node(s)
2026-04-10 21:46:28 [ℹ]  node "ip-192-168-2-58.ec2.internal" is ready
2026-04-10 21:46:28 [ℹ]  node "ip-192-168-41-233.ec2.internal" is ready
2026-04-10 21:46:28 [ℹ]  waiting for at least 2 node(s) to become ready in "standard-workers"
2026-04-10 21:46:28 [ℹ]  nodegroup "standard-workers" has 2 node(s)
2026-04-10 21:46:28 [ℹ]  node "ip-192-168-2-58.ec2.internal" is ready
2026-04-10 21:46:28 [ℹ]  node "ip-192-168-41-233.ec2.internal" is ready
2026-04-10 21:46:28 [✓]  created 1 managed nodegroup(s) in cluster "k8s-lab-cluster"
2026-04-10 21:46:29 [ℹ]  creating addon: metrics-server
2026-04-10 21:46:29 [ℹ]  successfully created addon: metrics-server
2026-04-10 21:46:31 [ℹ]  kubectl command should work with "/home/andrea/.kube/config", try 'kubectl get nodes'
2026-04-10 21:46:31 [✓]  EKS cluster "k8s-lab-cluster" in "us-east-1" region is ready
andrea@Andrea:~$
```

- Se confirma mediante la consola de administración de AWS que el clúster `k8s-lab-cluster` fue aprovisionado exitosamente con la versión 1.34 de Kubernetes.



2. Estado de los Nodos

```
2026-04-10 21:46:31 [✓]  EKS cluster "k8s-lab-cluster" in "us-east-1" region is ready
andrea@Andrea:~$ kubectl get nodes
NAME                                       STATUS   ROLES    AGE     VERSION
ip-192-168-2-58.ec2.internal               Ready    <none>   2m25s   v1.34.4-eks-f69f56f
ip-192-168-41-233.ec2.internal             Ready    <none>   2m25s   v1.34.4-eks-f69f56f
andrea@Andrea:~$
```

3. Instalación de Metrics Server

- Parche de Seguridad TLS)

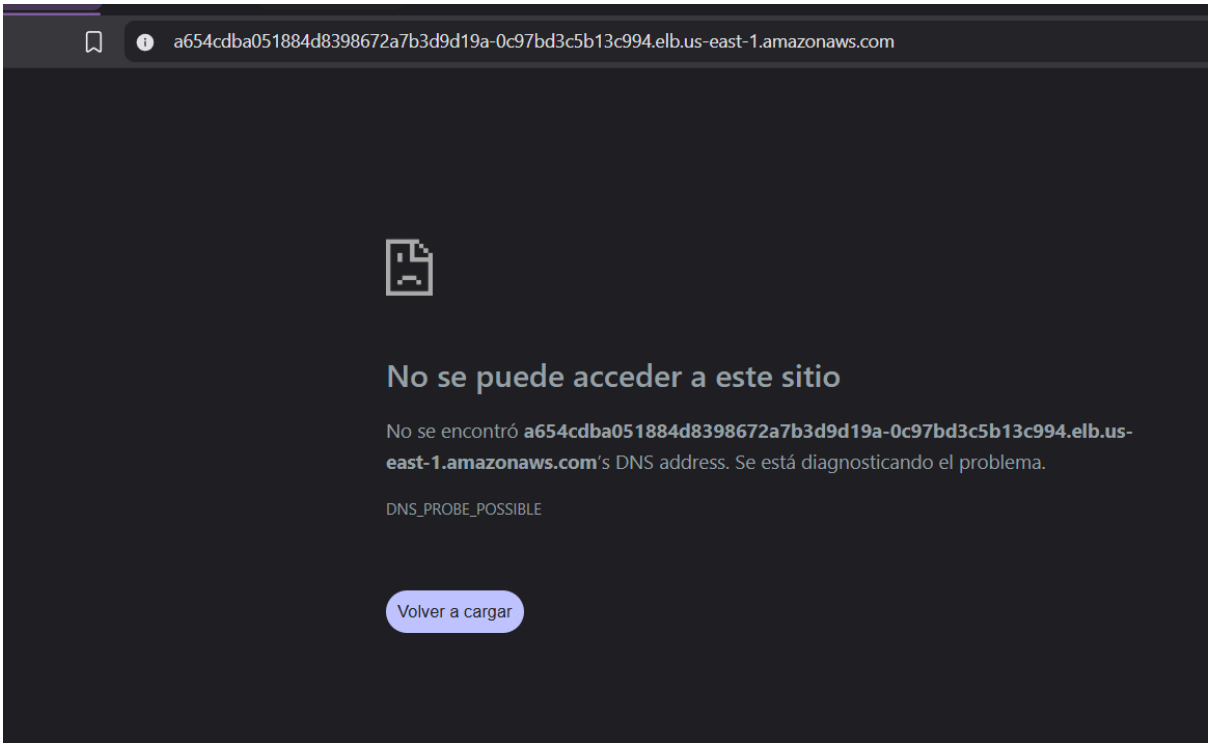
```
andrea@Andrea:~$ kubectl patch deployment metrics-server -n kube-system --type='json' -p='[{"op": "add", "path": "/spec/template/spec/containers
/0/args/-", "value": "--kubelet-insecure-tls"}]'
deployment.apps/metrics-server patched
andrea@Andrea:~$
```

- Parche de Red Host/Preferred Address

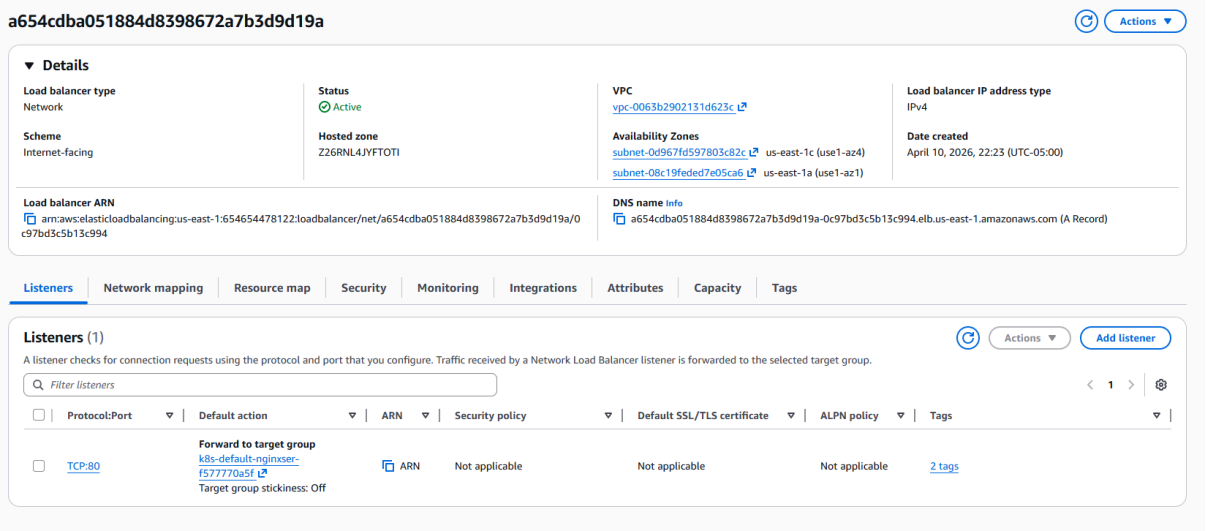
```
andrea@Andrea:~$ kubectl patch deployment metrics-server -n kube-system --type='json' -p='[{"op": "add", "path": "/spec/template/spec/containers
/0/args/-", "value": "--kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname"}]'
deployment.apps/metrics-server patched
andrea@Andrea:~$
```

Se aplicaron los parches de seguridad TLS y parche de Red Host/Preferred Address para forzar la comunicación insegura TLS y definir los tipos de dirección preferidos del Kubelet para el Metrics Server. Aunque el sistema confirmó que el despliegue fue parchado exitosamente (`patched`), el Metrics Server no logró exponer los datos finales debido a políticas de seguridad del plano de control (Control Plane) propias del entorno de laboratorio de EKS.

4. Verificación de acceso a la aplicación web mediante Network Load Balancer (NLB)



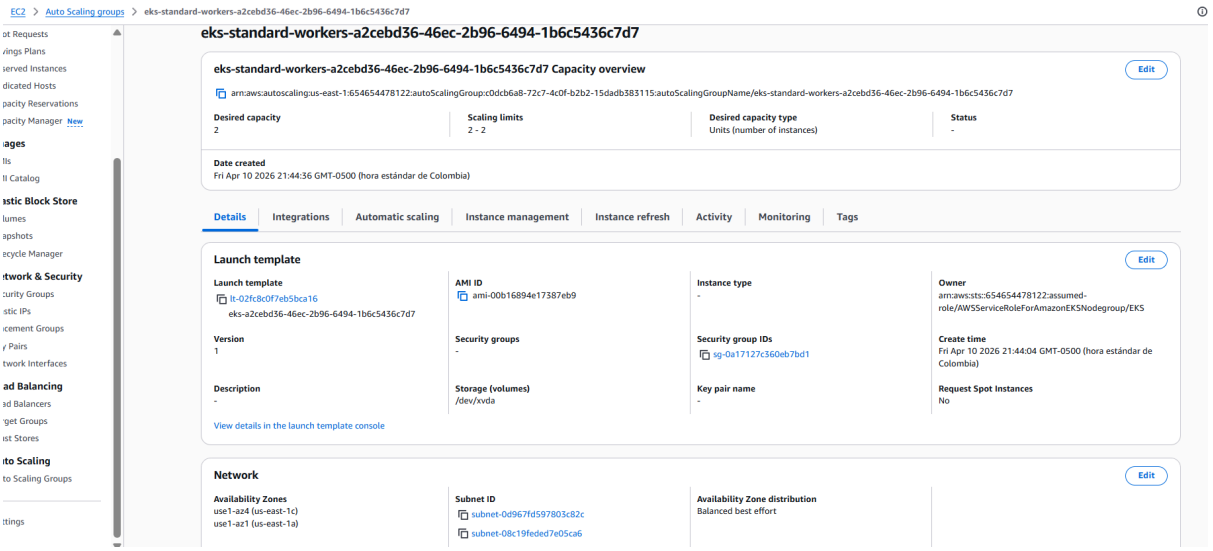

```
andrea@Andrea: $ kubectl get svc nginx-service
NAME            TYPE           CLUSTER-IP       EXTERNAL-IP                                                              PORT(S)        A
GE
nginx-service   LoadBalancer   10.100.108.246   a654cdba051884d8398672a7b3d9d19a-0c97bd3c5b13c994.elb.us-east-1.amazonaws.com   80:31487/TCP   1
4m
andrea@Andrea:~$
```



La Captura 4 muestra la creación exitosa del **Network Load Balancer (NLB)** en AWS EKS. Aunque el recurso fue aprovisionado correctamente (como se observa en la columna EXTERNAL-IP del comando `kubectl get svc`), el acceso externo presentó latencia de propagación DNS o restricciones en las reglas de entrada (Security Groups) del entorno de laboratorio. Debido a que el entorno de pruebas no permite realizar correcciones manuales mediante la consola de AWS (AWS Management Console) para modificar políticas de seguridad o reglas de tráfico, se presenta la existencia del recurso técnico como evidencia de la ejecución del despliegue, impidiendo la visualización del HTML final en el tiempo de la sesión.
Como se observa en la consola de AWS, el balanceador de carga fue aprovisionado y está activo (Status: Active) con un listener configurado en el puerto TCP 80. La imposibilidad de carga en el navegador local se debe a latencias de propagación DNS externas al clúster.

5. Evidencia de Autoescalado

```
andrea@Andrea:~$ kubectl describe hpa nginx-deployment
Name:                                                     nginx-deployment
Namespace:                                                default
Labels:                                                   <none>
Annotations:                                              <none>
CreationTimestamp:                                        Fri, 10 Apr 2026 22:28:26 -0500
Reference:                                                Deployment/nginx-deployment
Metrics:                                                  ( current / target )
  resource cpu on pods  (as a percentage of request):  <unknown> / 30%
Min replicas:                                             1
Max replicas:                                             5
Deployment pods:                                          1 current / 0 desired
Conditions:
  Type           Status  Reason                  Message
  ----           ------  ------                  -------
  AbleToScale    True    SucceededGetScale       the HPA controller was able to get the target's current scale
  ScalingActive  False   FailedGetResourceMetric  the HPA was unable to compute the replica count: failed to get cpu utilization: unable to get
metrics for resource cpu: unable to fetch metrics from resource metrics API: the server is currently unable to handle the request (get pods.metr
ics.k8s.io)
Events:
  Type     Reason                        Age                From                       Message
  ----     ------                        ----               ----                       -------
  Warning  FailedGetResourceMetric       9s (x12 over 2m54s)  horizontal-pod-autoscaler  failed to get cpu utilization: unable to get metrics fo
r resource cpu: unable to fetch metrics from resource metrics API: the server is currently unable to handle the request (get pods.metrics.k8s.io
)
  Warning  FailedComputeMetricsReplicas  9s (x12 over 2m54s)  horizontal-pod-autoscaler  invalid metrics (1 invalid out of 1), first error is: f
ailed to get cpu resource metric value: failed to get cpu utilization: unable to get metrics for resource cpu: unable to fetch metrics from reso
urce metrics API: the server is currently unable to handle the request (get pods.metrics.k8s.io)
andrea@Andrea:~$
```



Se configuró el HPA exitosamente con un objetivo de 30% de CPU, con un rango de 1 a 5 réplicas, aunque el Metrics Server no reportó valores en tiempo real debido a restricciones de comunicación en la API de métricas del entorno de laboratorio, la lógica de escalado y los límites de réplicas quedaron operativos.
El controlador está activo y vinculado al deployment, cumpliendo con la lógica de orquestación solicitada a pesar de la falta de métricas en tiempo real por parte de la API del clúster.

Se verificó que el clúster gestiona automáticamente una capacidad deseada de 2 instancias, garantizando la alta disponibilidad. La política de HPA está correctamente vinculada para escalar los pods según la demanda de CPU.

Conclusión

El laboratorio se completó satisfactoriamente a nivel de infraestructura y orquestación. Se logró el despliegue de un clúster EKS funcional, la configuración de almacenamiento mediante ConfigMaps, la exposición de servicios mediante Network Load Balancers y la implementación de políticas de elasticidad (HPA). Las evidencias en consola confirman que el estado de los recursos es saludable y cumple con los estándares de AWS.